MICRO CONTROLLER PRESENTATION

Group - 3

Name: Md. Ahsan Habib Sadik
ID: 2210976101

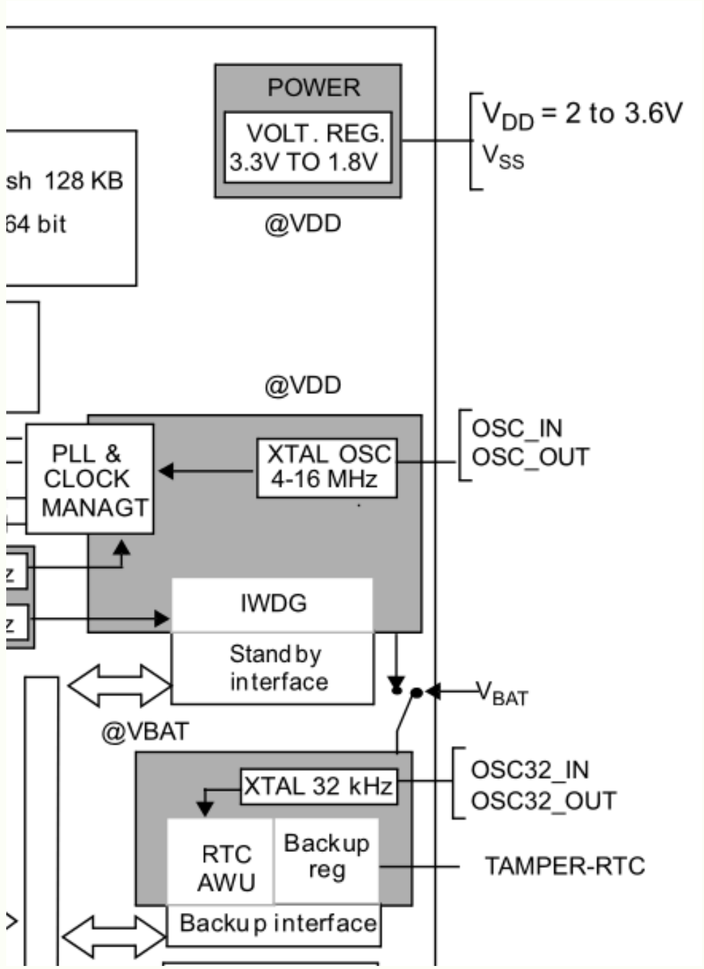
Name: Nur Alam Shikder
ID: 2210776127

Name: Shibli Noman Sarkar
ID: 2210776130

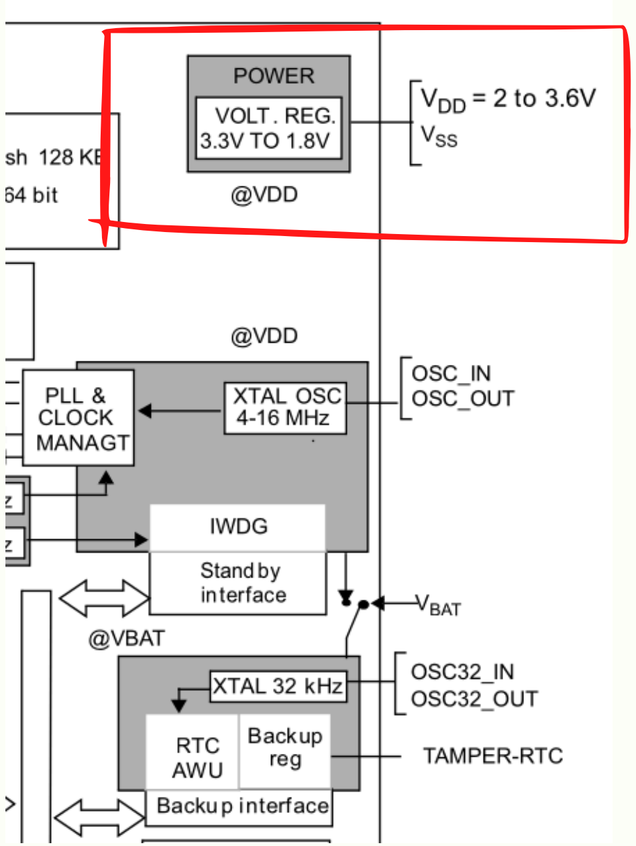
Name: Md. Mehedi Hasan Rafy
ID: 2210776132

POWER, CLOCK & BACKUP DOMAIN (STM32F103XX)

MD AHSAN HABIB SADIK



POWER & VOLTAGE REGULATOR



- INPUT: VDD = 2V TO 3.6V
- INTERNAL REGULATOR CONVERTS IT TO: 3.3V → 1.8V (CORE VOLTAGE)

WHAT IT DOES:

- SUPPLIES STABLE VOLTAGE TO CPU AND PERIPHERALS
- PROTECTS SYSTEM FROM UNSTABLE POWER

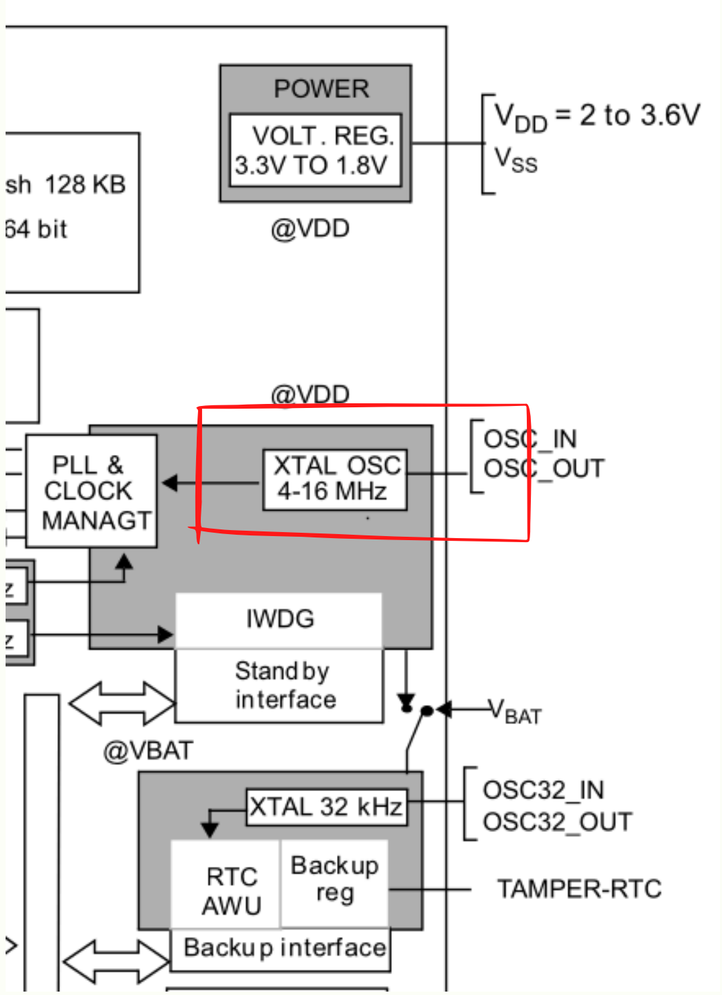
MAIN CRYSTAL OSCILLATOR (XTAL OSC 4–16 MHZ)

EXTERNAL CRYSTAL CONNECTED TO:

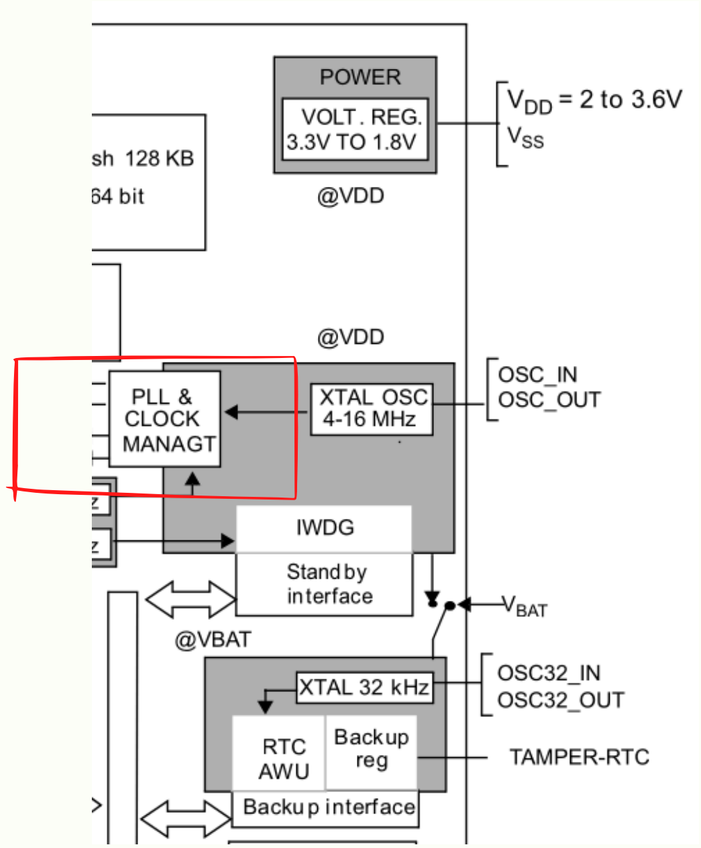
- OSC_IN
- OSC_OUT

WHAT IT DOES:

- GENERATES THE MAIN CLOCK SIGNAL
- USED BY:
  - CPU
  - PERIPHERALS
  - PLL (PHASE LOCKED LOOP)



PLL & CLOCK MANAGEMENT

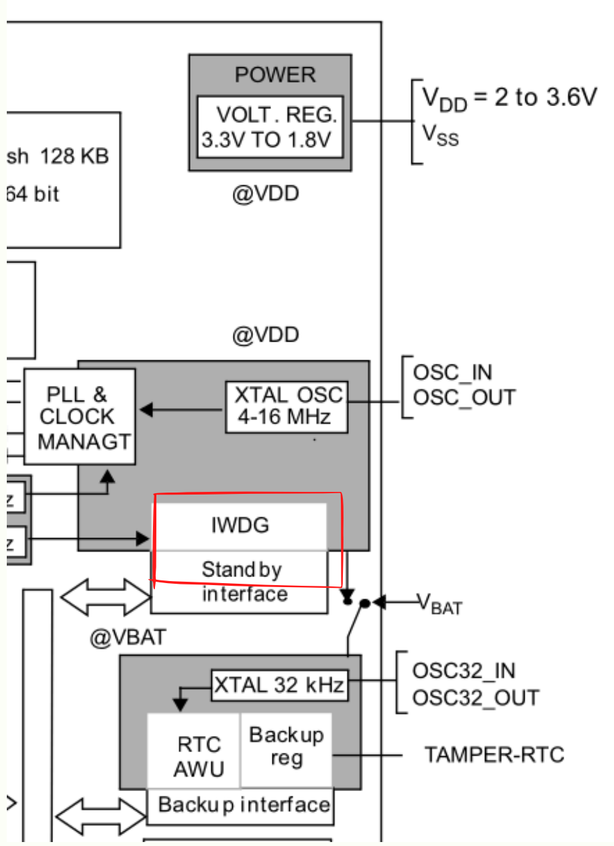


- TAKES CLOCK FROM OSCILLATOR
- MULTIPLIES FREQUENCY (E.G., 8 MHZ → 72 MHZ)

WHAT IT DOES:

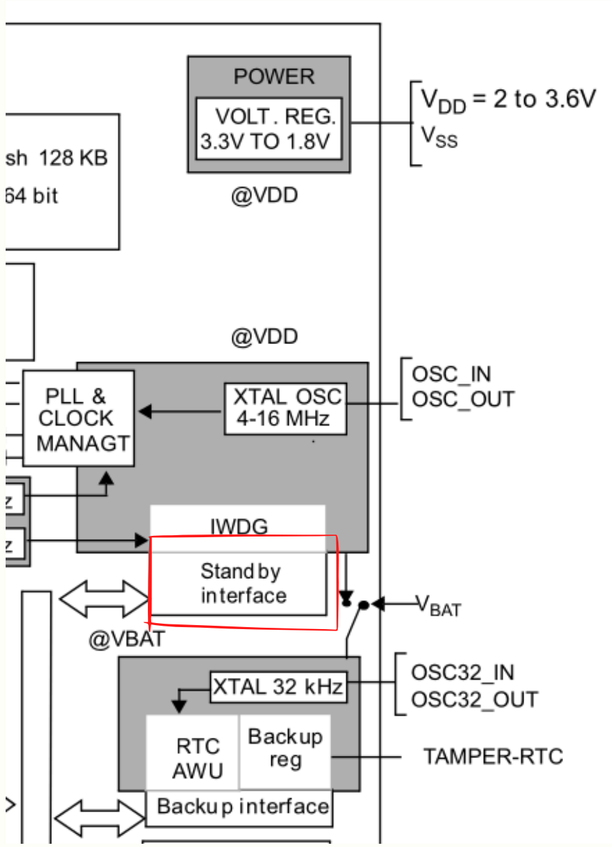
- INCREASES SYSTEM SPEED

IWDG (INDEPENDENT WATCHDOG):



- SAFETY TIMER
- RESETS MCU IF SOFTWARE HANGS OR ANY DEADLOCK OCCUR.

STANDBY INTERFACE



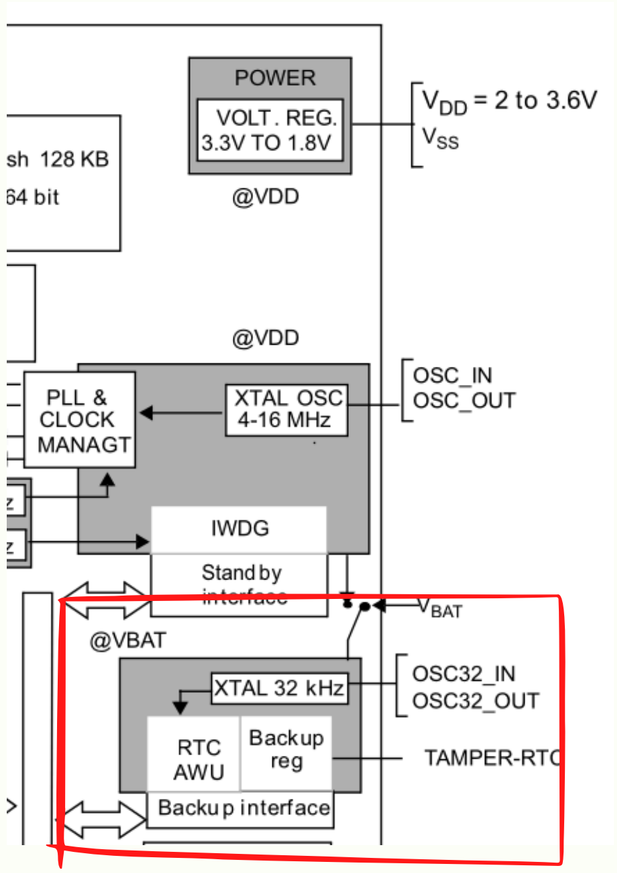
WHAT IT DOES:

- ENABLES LOW POWER MODE
- MCU CAN "SLEEP" TO SAVE POWER

USED IN:

- BATTERY DEVICES
- IOT SYSTEMS

Backup Domain (VBAT area)



THIS IS A SPECIAL LOW-POWER SECTION.

INCLUDES:

- VBAT (BATTERY INPUT)
- RTC (REAL-TIME CLOCK)
- BACKUP REGISTERS

WHAT IT DOES:

- KEEPS TIME RUNNING EVEN WHEN POWER IS OFF
- STORES SMALL IMPORTANT DATA

TIMER

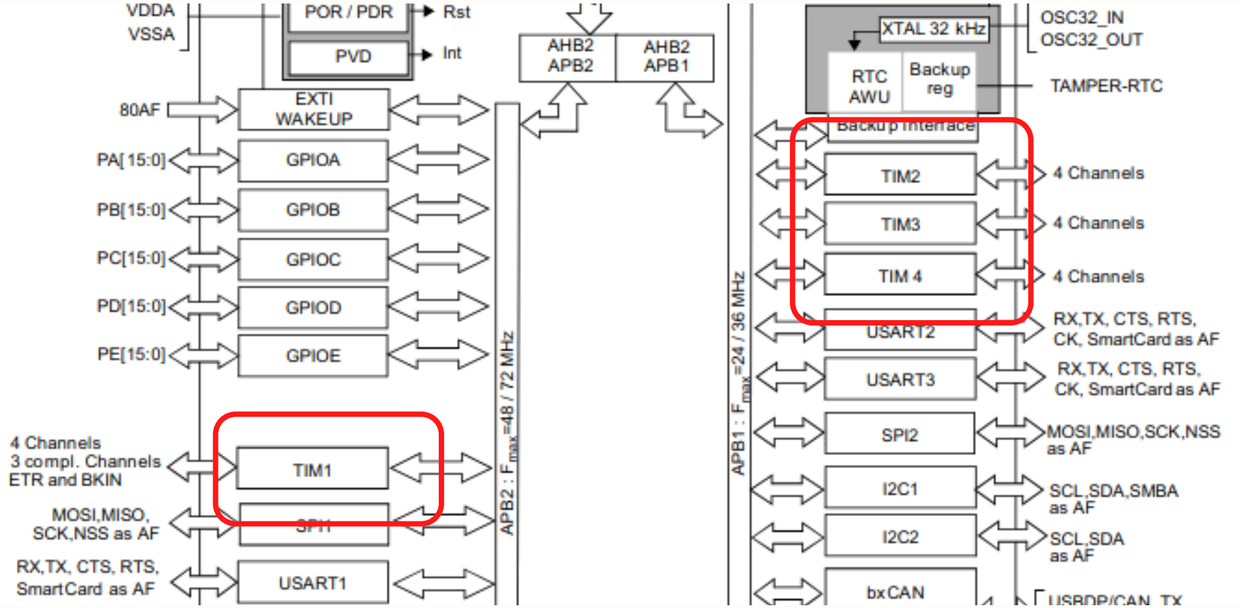
Nur Alam Shikder

INTRODUCTION TO STM32 TIMERS

TIMER IS A HARDWARE PERIPHERAL.

–COUNTS CLOCK PULSES.

–GENERATES DELAY, PWM, INTERRUPT.



BASIC TIMER BLOCK DIAGRAM

TIM1 – ADVANCED CONTROL TIMER

16-BIT ADVANCED TIMER

- HAS COMPLEMENTARY PWM (CHX & CHXN)
- SUPPORTS DEAD-TIME INSERTION
- HAS BREAK INPUT FOR SAFETY SHUTDOWN
- MAINLY USED FOR MOTOR CONTROL & POWER ELECTRONICS

TIM2 – 32-BIT GENERAL PURPOSE TIMER

32-BIT COUNTER (VERY LARGE RANGE)

- BEST FOR LONG DELAY APPLICATIONS
- SUPPORTS PWM, INPUT CAPTURE, OUTPUT COMPARE
- USED FOR PRECISE TIME MEASUREMENT

TIM3 & TIM4: 16-BIT GENERAL PURPOSE TIMERS

16-BIT COUNTERS

1. USED FOR PWM GENERATION
2. USED FOR LED DIMMING & SERVO CONTROL
3. GENERATE PERIODIC INTERRUPT
4. TIM3 AND TIM4 ARE ALMOST IDENTICAL

EXAMPLE: 1 KHZ PWM GENERATION

SYSTEM CLOCK = 72 MHZ

- PSC = 71 → COUNTER CLOCK = 1 MHZ
- ARR = 999 → FREQUENCY = 1 KHZ
- CCR = 500 → 50% DUTY CYCLE

COMPARISON SUMMARY

- TIM1 → ADVANCED MOTOR CONTROL TIMER
- TIM2 → 32-BIT LONG COUNTER TIMER
- TIM3 → 16-BIT GENERAL PURPOSE TIMER
- TIM4 → SAME AS TIM3 (DIFFERENT PINS)

SPI - SERIAL PERIPHERAL INTERFACE

Shibli Noman Sarkar

SPI (Serial Peripheral Interface) is a hardware peripheral built inside STM32 microcontrollers such as STM32F103C8T6.

It is used for high-speed serial communication between the microcontroller and external devices.

Examples of devices connected using SPI:

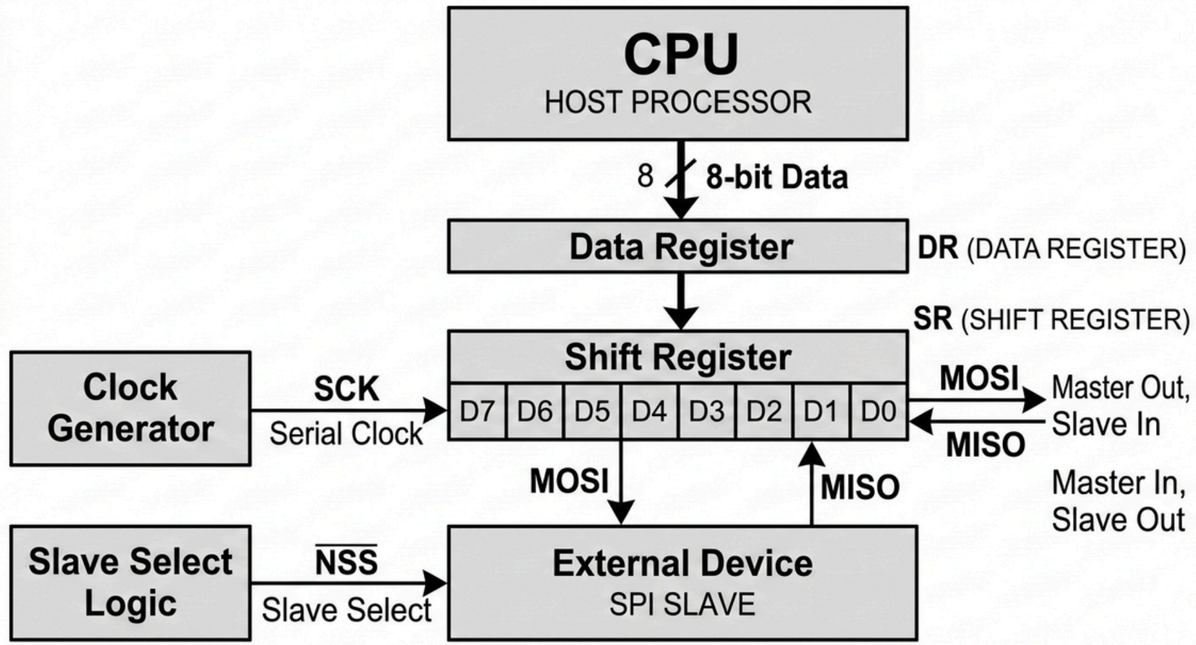
- Sensors
- SD Card
- LCD Display
- Flash Memory
- ADC / DAC

The Data Pathway (Top to Bottom)

CPU: The brain of the operation. It decides what data needs to be sent. However, the CPU is too fast to talk directly to external devices bit-by-bit, so it drops a whole byte (e.g., 01101001) into the next stage.

Data Register (DR): Think of this as a "holding bay" or buffer. It stores the byte from the CPU so the CPU can go back to other tasks while the hardware handles the actual transmission.

Shift Register (SR): This is the heart of the SPI. It takes the solid block of 8 bits from the Data Register and "shifts" them out one at a time (serially). Simultaneously, as it pushes a bit out, it pulls a new bit in from the external device.



Signal	Name	Function
MOSI	Master Out Slave In	The "Exit" ramp. Data travels from your Shift Register to the External Device.
MISO	Master In Slave Out	The "Entrance" ramp. Data travels from the External Device back into your Shift Register.
SCK	Serial Clock	The "Metronome." Generated by the **Clock Generator**, it tells both devices exactly when to push or pull a bit.
NSS / CS	Slave Select	The "Doorbell." Since a CPU can be connected to many devices, this line tells one specific **External Device** to wake up and listen.

How the "Exchange" Works:

SPI is full-duplex, meaning data goes out and comes in at the exact same time. Imagine two people standing in a line, each holding a bucket of 8 balls. Every time the Clock (SCK) ticks, the person on the left (CPU) hands one ball to the right, and the person on the right (External Device) hands one ball to the left. After 8 ticks, they have completely swapped buckets.

Summary of the Workflow:

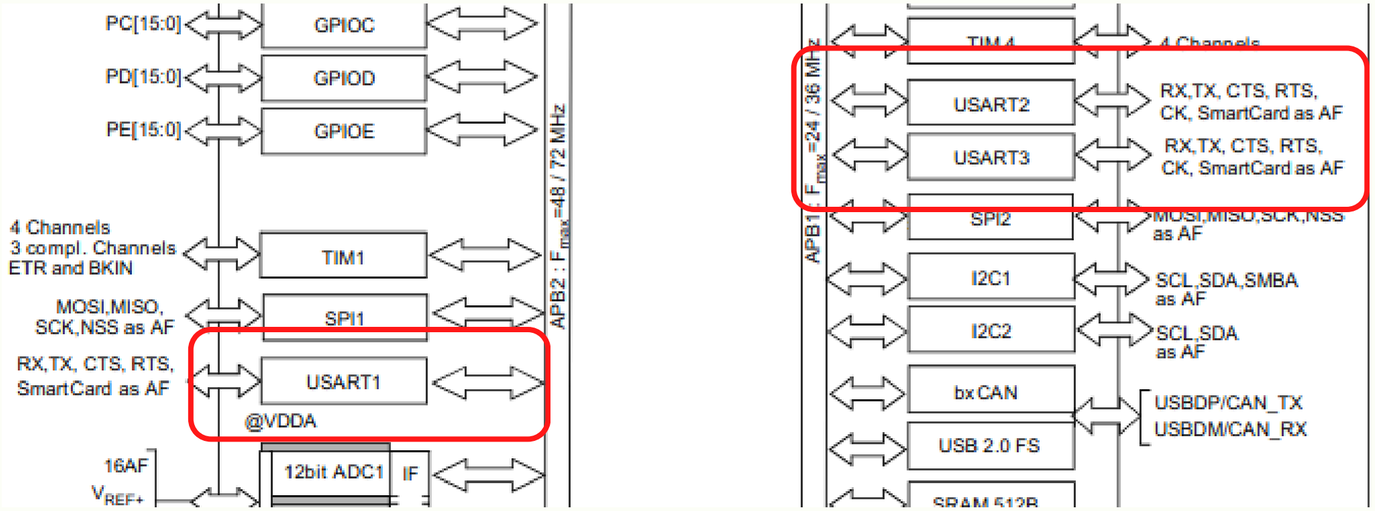
- Select: The CPU pulls the NSS line low to "grab the attention" of the external device.
- Load: The CPU moves a byte into the Data Register.
- Shift: The Clock Generator starts pulsing SCK. For every pulse, one bit moves out via MOSI and one bit moves in via MISO.
- Finish: Once the Shift Register is empty (and refilled with new data), the CPU reads the result and releases the NSS line.

USART - UNIVERSAL SYNCHRONOUS/ASYNCHRONOUS RECEIVER TRANSMITTER

Md. Mehedi Hasan Rafy

INTRODUCTION

- WHAT IS USART?
  - A HARDWARE PERIPHERAL BUILT INSIDE STM32F103X8 / STM32F103XB, USED FOR SERIAL COMMUNICATION.



- STM32F103 PROVIDES - USART1, USART2, USART3

WHY MULTIPLE USART?

- ALLOWS COMMUNICATION WITH MULTIPLE DEVICES SIMULTANEOUSLY
  - USART1 → PC DEBUGGING
  - USART2 → GSM MODULE
  - USART3 → GPS MODULE
- ALL OPERATE INDEPENDENTLY.

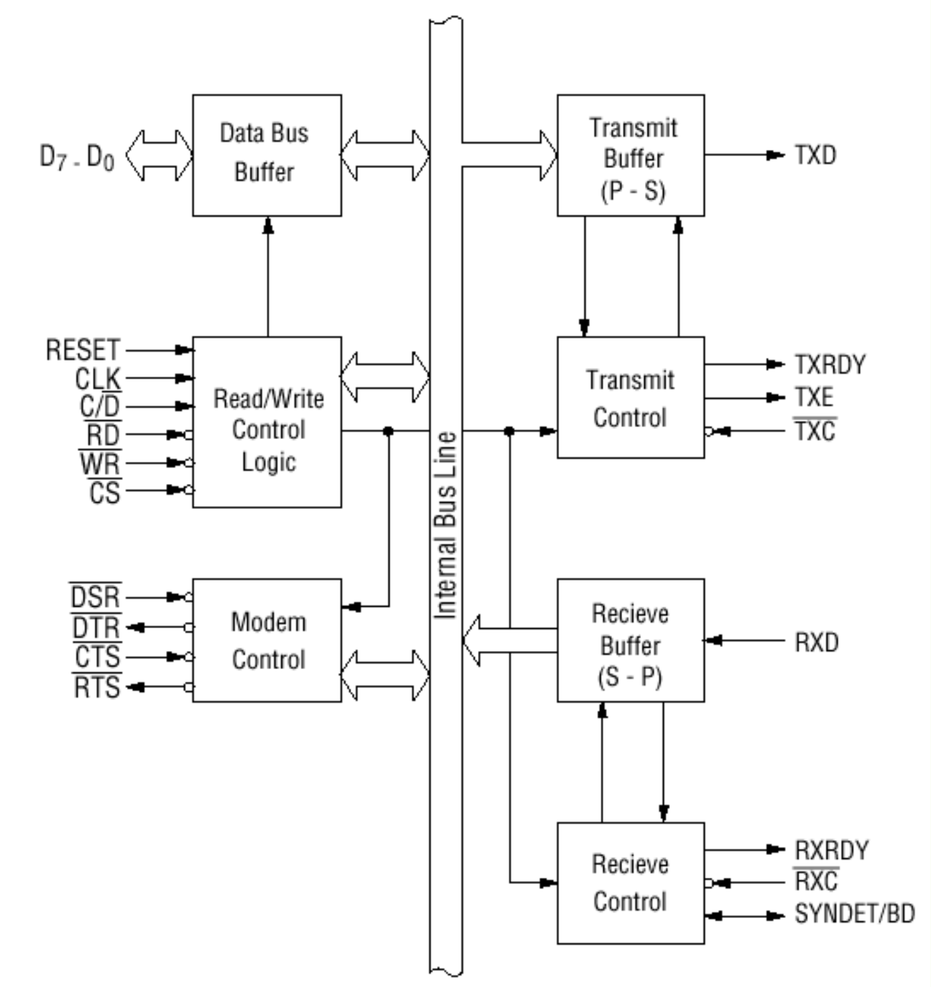
ROLE OF USART IN STM32 SYSTEM

1. • ACTS AS A COMMUNICATION BRIDGE BETWEEN MCU AND THE EXTERNAL WORLD
2. • HANDLES SERIAL TIMING AUTOMATICALLY
3. • REDUCES CPU WORKLOAD
4. • ENABLES REAL-TIME DATA EXCHANGE
5. • ESSENTIAL FOR DEBUGGING AND FIELD DEPLOYMENT

USART ARCHITECTURE

INTERNAL STRUCTURE

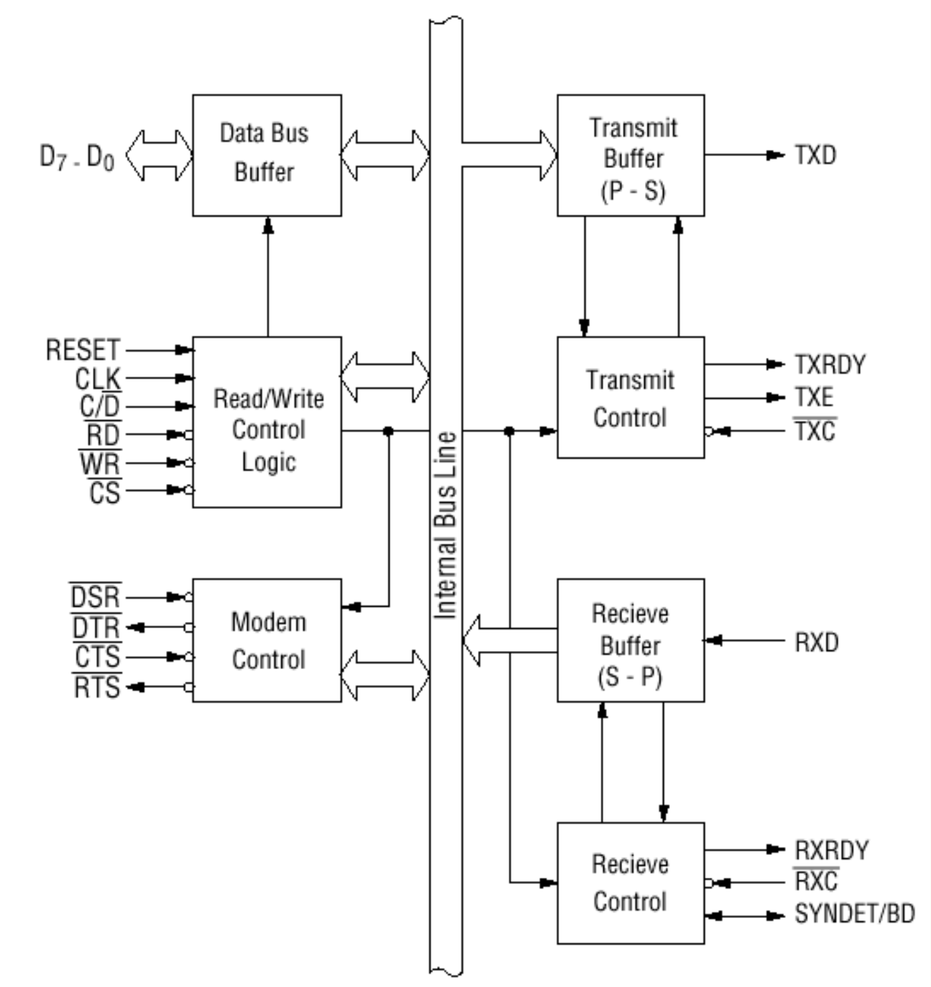
1. DATA BUS BUFFER - ACTS AS AN INTERFACE BETWEEN THE CPU DATA BUS AND THE INTERNAL BUS.
2. READ/WRITE CONTROL LOGIC - CONTROLS OVERALL OPERATION. IT -
   A. SELECTS REGISTERS - DATA BUFFER REGISTER, CONTROL REGISTER, STATUS REGISTER
   B. OPERATES BASED ON CONTROL SIGNALS - RD (READ), WR (WRITE), C/D (CONTROL/DATA)
3. TRANSMIT SECTION -
   A. TRANSMIT BUFFER - TEMPORARY STORAGE FOR PARALLEL DATA FROM THE CPU.
   B. TRANSMIT CONTROL - CONTROLS THE TRANSMISSION PROCESS.
   C. TXD - TRANSMIT DATA



USART ARCHITECTURE

INTERNAL STRUCTURE

4. RECEIVE SECTION -
   A. RECEIVE BUFFER - STORES RECEIVED SERIAL DATA TEMPORARILY.
   B. RECEIVE CONTROL - CONTROLS THE RECEIVING PROCESS.
   C. RXD - RECEIVE DATA
5. MODEM CONTROL SECTION -
   A. USED WHEN COMMUNICATING THROUGH A MODEM.
   B. PROVIDES HARDWARE FLOW CONTROL.



THANK YOU